

Consumindo Dados de um Banco de Dados Relacional

Objetivo da Aula

Utilizar o Sequelize para consultar e gerenciar os dados de uma base MySQL, incluindo o uso de transações e de cache de entidades.

Apresentação

É fundamental, para qualquer desenvolvedor, saber como lidar com as bases de dados relacionais, a partir da plataforma de programação, já que os sistemas cadastrais irão utilizar os bancos de dados como repositórios das entidades.

Aqui, veremos como utilizar o Sequelize para implementar inclusão, consulta, alteração e exclusão de registros no MySQL, ou seja, iremos implementar as operações de persistência de um CRUD. Após programarmos nosso gerenciador de persistência, poderemos utilizá-lo em qualquer sistema NodeJS.

Outro assunto importante é o gerenciamento de transações no Sequelize, já que a execução de alterações em lote pode ser difícil de desfazer, no caso de ocorrência de erro, mas, com o uso de transações, torna-se uma operação simples.

Finalmente, veremos como o redis pode ser utilizado para oferecer um cache de entidades em memória, melhorando bastante o tempo de resposta de nosso servidor para algumas consultas específicas.

1. Operações de Consulta e Inclusão

Vamos implementar a consulta e a inclusão de produtos no MySQL, através do Sequelize, e, para tal, você deve criar um diretório vazio, abrir no VS Code e configurar o projeto por meio dos comandos apresentados a seguir.

```
npm init -y
npm install sequelize mysql2
```



Destaque

Para que os exemplos desta aula funcionem, você precisará de uma tabela **produtos**, com os campos **codigo**, **nome** e **quantidade**. O código deve ser inteiro e autoincrementado, a quantidade é inteira, e o nome é texto.

Após instalar as bibliotecas, defina a conexão com o banco, de acordo com o código seguinte, no arquivo **db.js**.

```
import { Sequelize } from "sequelize";
const sequelize = new Sequelize("loja_node", "root", "adm12",
    {dialect: "mysql", host: "127.0.0.1"});
export default sequelize;
```

O passo seguinte é a definição da entidade, no arquivo **model-produto.js**.

```
import { Sequelize } from "sequelize";
import db from "../db.js"; // Objeto sequelize

export default db.define("produto", {
    codigo: {type: Sequelize.INTEGER.UNSIGNED, primaryKey: true,
        autoIncrement: true, allowNull: false},
    nome: {type: Sequelize.STRING, allowNull: false},
    quantidade: {type: Sequelize.INTEGER, allowNull: true}
}, {timestamps: false}); // createdAt, updatedAt
```

Agora que nosso projeto foi configurado para lidar com a tabela de produtos, crie o arquivo **controller-produto.js**, no qual serão implementadas as operações que irão utilizar o Sequelize. Comece a codificação do arquivo com a listagem apresentada a seguir.

```
import ProdutoRepo from "../model-produto.js";
import {Op} from "sequelize";

export async function obterTodos() {
    return await ProdutoRepo.findAll();
}

export async function obterProduto(codigo) {
    return await ProdutoRepo.findByPk(codigo);
}
```

A função **obterTodos** utiliza o método **findAll**, do Sequelize, para recuperar todos os produtos da base de dados, enquanto **obterProduto** recupera apenas um produto, a partir de seu código, através de **findByPk**. Com isso, temos os dois tipos de consulta mais comuns em qualquer sistema cadastral.

Note que temos a inclusão do elemento **Op**, de Operator, que ainda não foi utilizado, mas que será explorado no passo seguinte. Adicione a função que é apresentada no fragmento de código ao arquivo **controller-produto.js**.

```
export async function obterNomeQtd(nome, quantidade) {
    return await ProdutoRepo.findAll({
        attributes: ['nome', 'quantidade'],
        where: {
            nome: { [Op.like]: `${nome}%` },
            quantidade: { [Op.gt]: quantidade }
        }
    });
}
```

A função **obterNomeQtd** utiliza novamente o método **findAll**, mas agora de forma parametrizada. Através de uma configuração em JSON, você pode definir quais os campos que serão retornados, em **attributes**, ou as restrições da pesquisa na cláusula **where**.

Em nosso exemplo, o elemento **Op** foi utilizado para definir duas condições, ao efetuar a pesquisa, em que temos os nomes iniciados pelo valor fornecido, via **Op.like**, e a quantidade equivalente à fornecida no parâmetro, via **Op.gt**. Existem muitos outros operadores disponíveis, permitindo efetuar consultas complexas, que serão traduzidas pelo Sequelize em comandos SQL.

Para implementar a inserção, acrescente a função seguinte ao controlador.

```
export async function criarProduto(nome, quantidade) {  
  const produto = await ProdutoRepo.create(  
    {nome: nome, quantidade: quantidade});  
  return produto.codigo;  
}
```

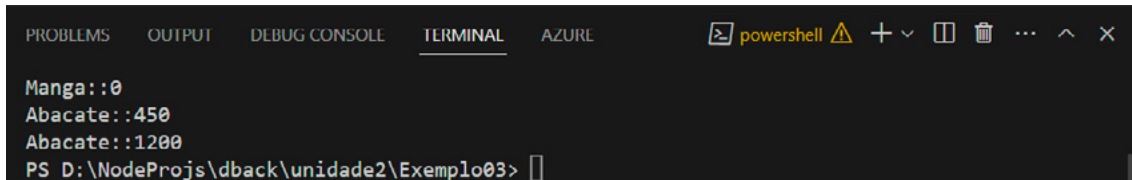
A função **criarProduto** utiliza o método **create**, de Sequelize, para incluir um produto na base de dados, sendo fornecidos os valores dos campos no formato JSON – à exceção do código, que é autoincrementado. Os valores são fornecidos através dos parâmetros da função, sendo retornado o código do novo produto.

Já podemos testar a inclusão e a consulta, por meio do código seguinte.

```
import {criarProduto, obterTodos}  
  from "./controller-produto.js";  
criarProduto("Abacate", 1200).then(  
  _ => obterTodos().then(  
    (produtos) => {  
      for(let p of produtos)  
        console.log(p.nome + "::" + p.quantidade)  
    }  
  )  
)
```

Executando o trecho de código anterior, ocorrerá a inserção do produto na tabela, e, após essa inclusão, o conjunto completo de produtos será recuperado, e cada um deles terá a impressão de seus campos no console.

Figura 1: Execução de inclusão e consult



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL AZURE powershell
Manga::0
Abacate::450
Abacate::1200
PS D:\NodeProjs\dback\unidade2\Exemplo03>
  
```

Fonte: Elaboração própria.

2. Operações de Alteração e Exclusão

De forma geral, alterações e exclusões são bastante semelhantes, pois, em ambos os casos, se não for definida uma restrição, todos os registros da tabela serão impactados. Pela configuração padrão do MySQL, os comandos **update** e **delete** são bloqueados quando não existe uma cláusula **where** no SQL.

Vamos começar pela alteração de uma entidade específica, acrescentando a função seguinte ao controlador.

```

export async function alterarProduto(codigo, nome, quantidade){
    let produto = await ProdutoRepo.findByPk(codigo);
    produto.nome = nome;
    produto.quantidade = quantidade;
    return await produto.save();
}
  
```

Na função **alterarProduto**, a entidade atual é obtida utilizando **findByPk**, seus campos são atualizados, e as alterações são gravadas no banco de dados com a utilização do método **save**. O retorno da função engloba os valores que foram alterados e diversos campos de status no formato JSON.

Outra forma de alterar os dados é através do método **update**, que permite alterar diversas entidades simultaneamente, como na função seguinte.

```
export async function zerarQtd(){
  const [updateRows] = await ProdutoRepo.update(
    {quantidade: 0},
    {where: {quantidade: null}}
  );
  return updateRows;
}
```

Ao utilizar **update**, na função **zerarQtd**, foram fornecidos os valores para os campos no formato JSON, através do primeiro parâmetro, enquanto o segundo apresenta a restrição no atributo **where**. A execução dessa operação irá atribuir o valor zero a todos os produtos que estiverem com quantidade nula.

Da mesma forma que no método **save**, o retorno de **update** envolve vários campos de status, nos quais um deles indica a quantidade de registros alterados. No caso, o campo é **updateRows**, que pode ser extraído da resposta com o uso de colchetes, sendo retornado pela função ao final.

A função para exclusão individual é apresentada na listagem seguinte e deve ser adicionada ao código do controlador.

```
export async function removerProduto(codigo) {
  let produto = await ProdutoRepo.findByPk(codigo);
  await produto.destroy();
}
```

De forma semelhante a **alterarProduto**, na função **removerProduto** é obtida a entidade atual, por meio de **findByPk**, com a passagem do código fornecido no parâmetro da função. Em seguida, o método **destroy**, da entidade, é utilizado para excluir o registro na base de dados.

Para a exclusão de múltiplos registros, o método **destroy** pode ser utilizado com um parâmetro, fornecendo a cláusula **where**.

```
export async function removerZerados(){
  return await ProdutoRepo.destroy(
    {where: {quantidade: 0}}
  );
}
```

A função **removerZerados** invoca o método **destroy**, fornecendo a restrição no formato JSON. Como resultado de sua execução, seriam removidos todos os registros com quantidade igual a zero, tendo como retorno o total de registros que foram excluídos.

Para restrições mais complexas, pode ser utilizado o elemento **Op**, como nos exemplos do quadro seguinte.

Quadro 1: Exemplos de utilização do elemento Op

Operadores	Grupo	Exemplos
Op.and, Op.or	Lógicos	[Op.or]: [{ a: 5 }, { b: 6 }] // (a==5) (b==6)
Op.eq, Op.ne, Op.is, Op.not	Básicos	a: {[Op.or]: {[Op.eq]:3,[Op.is]:null}} // (a==3) (a==null)
Op.gt, Op.gte, Op.lt, Op.lte	Aritméticos	createdAt: {[Op.gt]: new Date(new Date() - 3600000)} // Registros criados na última hora
Op.in, Op.notIn	Conjuntos	a: {[Op.in]: [1,3,5]} // (a==1) (a==3) (a==5)
Op.like, Op.notLike	Texto	nome: {[Op.like]: 'Jo%'} // nome começa com "Jo" (João, José)

Fonte: *Elaboração própria.*

3. Gerenciamento de Transações

Em termos de bancos de dados, uma transação representa um conjunto de operações para manipulação de registros que são tratadas em grupo, podendo ocorrer a confirmação ou rejeição do lote. Com o uso de transações, operações complexas podem ser implementadas com grande facilidade e segurança, pois não temos que desfazer ações anteriores manualmente após uma falha.

Toda transação deve ser atômica, consistente, isolada e durável, o que ficou conhecido pelo acrônimo **ACID**. Essas características significam o seguinte:

- **Atomicidade:** o grupo de operações é tratado como uma unidade, e todas elas devem ser concluídas para que o grupo seja efetivado;
- **Consistência:** todas as restrições de campo e chaves estrangeiras devem ser obedecidas ao executar as operações;
- **Isolamento:** os resultados parciais das operações não podem ser visualizados por outras conexões ao banco de dados;
- **Durabilidade:** quando a transação é efetivada, todas as alterações efetuadas são permanentes.

Uma transação é confirmada através do comando **commit**, que efetiva todas as operações na base de dados. Por outro lado, o comando **rollback**, que deve ser utilizado após uma falha, desfaz todas as operações que foram executadas desde o início da transação.

Quando você utiliza Sequelize, pode trabalhar com as transações gerenciadas ou **Managed Transactions**, nas quais não é necessário utilizar os comandos commit e rollback, e com as não gerenciadas ou **Unmanaged Transactions**, que devem ter a confirmação ou rejeição controlada pelo desenvolvedor.

A listagem seguinte apresenta um exemplo de **Managed Transaction**.

```
const {Pessoa, sequelize} = require('./models')
const incluirPessoa = async () => {
  try { // Managed Transaction
    const p = await sequelize.transaction(async (t) => {
      const p1 = await Pessoa.create({
        nome: 'Ana', sobrenome: 'Clara'
      }, {transaction: t});
      return p1;
    });
    return p; // Neste ponto ocorre commit automático
  } catch(error) {} // No caso de erro, rollback automático
}
incluirPessoa().then((p)=>console.log(JSON.stringify(p)));
```

Para a execução do exemplo, deve ser considerada uma entidade **Pessoa**, criada através do **Sequelize CLI**. O comando **transaction** é invocado, a partir do objeto **Sequelize**, sendo fornecida uma **callback**, em que **t** é a transação iniciada, dentro de uma estrutura com **try** e **catch**.

Note que a chamada para o método **create**, da entidade, recebe um segundo parâmetro, indicando a participação na transação. Como ela é do tipo gerenciada, se um erro for gerado nas operações participantes, o **rollback** é automático, e, se não ocorrerem erros até o final da callback, o **commit** será executado.

Figura 2: Execução de Managed Transaction

```
Executing (59bf6580-e60c-4e1e-bd75-c86d7ebbf07d): START TRANSACTION;
Executing (59bf6580-e60c-4e1e-bd75-c86d7ebbf07d): INSERT INTO `Pessoas` (`id`,`nome`,`sobrenome`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?);
Executing (59bf6580-e60c-4e1e-bd75-c86d7ebbf07d): COMMIT;
{"id":5,"nome":"Ana","sobrenome":"Clara","updatedAt":"2023-08-20T08:43:43.009Z","createdAt":"2023-08-20T08:43:43.009Z"}
```

Fonte: Elaboração própria.

A próxima listagem demonstra o uso de **Unmanaged Transaction**.

```
const {Pessoa, sequelize} = require('./models')
const incluirPessoa = async () => {
  const t = await sequelize.transaction();
  try { // Unmanaged Transaction
    const p = await Pessoa.create({
      nome: 'Joao', sobrenome: 'Paulo'
    }, {transaction: t});
    await t.commit();
    return p;
  } catch (error) {
    await t.rollback();
  }
}
incluirPessoa().then((p)=>console.log(JSON.stringify(p)));
```

Ao contrário do modelo anterior, aqui a transação é obtida na variável `t`, sem o uso de uma callback, e os métodos **commit** e **rollback** são invocados de forma explícita, oferecendo ao desenvolvedor maior controle.

Uma ferramenta interessante no Sequelize é o método **afterCommit**, que é definido como um **hook** (gancho) executado após a confirmação da transação. Ele pode ser utilizado, por exemplo, para gravar em um arquivo de log ou para enviar um e-mail confirmando o cadastro.

```
const t = await sequelize.transaction({
  isolationLevel: IsolationLevel.SERIALIZABLE
});
t.afterCommit(() => {console.log("Sucesso!");});
// A callback é executada após o commit
// Operações no banco de dados via Sequelize
await t.commit();
```

Além do **afterCommit**, temos os hooks **afterRollback**, que é executado após o rollback, e **afterTransaction**, que executa ao final da transação, independentemente do uso de commit ou rollback.

Ao executar uma transação, os registros alterados podem ser bloqueados para leitura por outras transações. Nesse contexto, uma configuração importante é o nível de isolamento, que poderá assumir os seguintes valores:

- **IsolationLevel.READ_UNCOMMITTED**: concorrência otimista, quando uma transação pode ler registros que são alterados por outras transações ainda não finalizadas;
- **IsolationLevel.READ_COMMITTED**: modo padrão para a maioria dos bancos, que permite a leitura apenas de registros com transações finalizadas;
- **IsolationLevel.REPEATABLE_READ**: gerador de bloqueios tanto nos registros lidos quanto para os modificados na transação;
- **IsolationLevel.SERIALIZABLE**: nível mais restritivo, gerando bloqueios para qualquer operação, inclusive inserção em uma tabela consultada em outra transação.

4. Cache de Entidades

Com o uso de cache, é possível diminuir o tempo de resposta em consultas sucessivas aos mesmos objetos. A técnica é aplicável tanto para tabelas que não se modificam constantemente, como países, cidades e estados, quanto para registros individuais.

Inicialmente precisamos do **redis** em execução no **Docker**, para funcionar como cache de entidades em memória. Caso ainda não tenha a imagem, efetue a busca por redis, no Docker Desktop, e execute um container.



Destaque

Para que os códigos desta aula funcionem, você precisará do projeto do primeiro módulo implementado, englobando a consulta de produtos.

Acrescente a dependência do redis ao seu projeto.

```
npm install redis;
```

Após incluir a dependência, inicie a codificação do exemplo, de acordo com a listagem apresentada a seguir.

```
import redis from "redis";
import {obterProduto} from "../controller-produto.js"
let redisClient;
(async () => {
  redisClient = redis.createClient();
  redisClient.on("error",
    (error) => console.error(`Error: ${error}`));
  redisClient.on('connect',
    () => console.log("REDIS: Conectado"));
  await redisClient.connect();
})();
```

No passo inicial, temos a criação de um cliente para o redis, com os eventos **error** e **connect** configurados, e a execução da conexão em seguida. Esse estilo de programação permite a execução automática de uma callback assíncrona, algo bastante útil quando precisamos utilizar `await`.

Continue a codificação, acrescentando o conteúdo da listagem seguinte.

```
const obter = async (codigo) => {
  let isCached = false, result;
  const cacheResults = await redisClient.get(`Prod${codigo}`);
  if(cacheResults){
    isCached = true; result = JSON.parse(cacheResults);
  } else {
    result = await obterProduto(codigo);
    await redisClient.set(
      `Prod${codigo}`, JSON.stringify(result));
  }
  return {fromCache: isCached, produto: result};
}
```

A função `obter` se inicia com a tentativa de obtenção da entidade, identificada pela concatenação de “Prod”, com o código da entidade, no **redis**. Se conseguir obter, a entidade é retornada, e o atributo **isCached** recebe **true**.

Quando a entidade não está no redis, ela é consultada via **Sequelize**, sendo adicionada, em seguida, ao redis. Nesse caso, o atributo **isCached** vale **false**.

Ao final, é retornado um resultado em **JSON**, informando se a informação veio do cache e os dados da entidade. Para efetuar um teste, complemente o código do exemplo com os comandos da listagem seguinte.

```
obter(1).then((x)=>console.log(JSON.stringify(x)));
obter(2).then((x)=>console.log(JSON.stringify(x)));
obter(1).then((x)=>console.log(JSON.stringify(x)));
```

O resultado de duas execuções sucessivas do exemplo seria a obtenção a partir do cache na segunda execução.

Figura 3: Consulta com uso de cache

```
PS D:\NodeProjs\dback\unidade2\Exemplo03> node index2.js
REDIS: Conectado
{"fromCache":false,"produto":{"codigo":1,"nome":"Laranja","quantidade":500}}
{"fromCache":false,"produto":{"codigo":2,"nome":"Banana","quantidade":1001}}
{"fromCache":false,"produto":{"codigo":1,"nome":"Laranja","quantidade":500}}
PS D:\NodeProjs\dback\unidade2\Exemplo03> node index2.js
REDIS: Conectado
{"fromCache":true,"produto":{"codigo":1,"nome":"Laranja","quantidade":500}}
{"fromCache":true,"produto":{"codigo":2,"nome":"Banana","quantidade":1001}}
{"fromCache":true,"produto":{"codigo":1,"nome":"Laranja","quantidade":500}}
```

Fonte: Elaboração própria.



Destaque

Para um cache realmente funcional, as operações de update e delete devem buscar, no cache, pela existência da entidade e efetuar a alteração ou remoção necessária, refletindo o estado corrente no banco de dados.

Considerações Finais da Aula

Com base no Sequelize e MySQL, fomos capazes de definir um ambiente de persistência completo, envolvendo todas as operações necessárias para os sistemas cadastrais comuns. Algo interessante é o fato de que, ao concentrar as operações em controladores específicos, podemos trabalhar em camadas, o que facilita bastante a manutenção com o crescimento do sistema.

Além de ver todas as operações comuns de gerenciamento de dados, vimos como garantir a consistência de operações em lote, com a utilização de transações, bem como que o comportamento das transações poderá ser diferente, em cada contexto, de acordo com o nível de isolamento adotado.

Não menos importante, utilizamos o redis para criar um cache de entidades, visando aumentar a eficiência do sistema, com um gerenciamento muito simples, via Docker. Ao nível do código JavaScript, o processo se resumiu à conexão com o redis e ao gerenciamento de dados no formato chave-valor.

Materiais Complementares

Artigo

Transactions

2023, Zoé. Sequelize.

O artigo faz parte da documentação oficial do Sequelize, com bons exemplos para elucidar o uso de transações na plataforma.

Link para acesso: <https://sequelize.org/docs/v6/other-topics/transactions/> (acesso em 12 set. 2023.)

Artigo

How To Implement Caching in Node.js Using Redis

2022, Digital Ocean.

O artigo da Digital Ocean demonstra como é a criação de um sistema de cache para o NodeJS, utilizando o servidor redis como repositório em memória.

Link para acesso: <https://www.digitalocean.com/community/tutorials/how-to-implement-caching-in-node-js-using-redis> (acesso em 12 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidadePaginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 12 set. 2023.